Documento de Projeto Arquitetural.

Aplicação da Arquitetura Hexagonal no SIGER

A **Arquitetura Hexagonal**, também conhecida como *Ports and Adapters* (Portas e Adaptadores), tem como principal objetivo isolar as regras de negócio da aplicação de tecnologias externas, como banco de dados, interfaces, serviços de e-mail e APIs.

No SIGER, essa arquitetura permite que as funcionalidades centrais do sistema funcionem sem depender diretamente de uma tecnologia específica.

1- Núcleo da aplicação

O núcleo da aplicação concentra as regras de negócio e os casos de uso do SIGER.

Ele pode ser dividido em duas partes:

- **Domínio:** contém as regras fundamentais do sistema, como verificar a disponibilidade de um equipamento, impedir reservas conflitantes, validar usuários e calcular penalidades por atraso.
- **Casos de uso:** coordenam as operações que os usuários podem realizar, como solicitar empréstimos, solicitar reservas, aprovar solicitações e registrar retiradas e devoluções.

Por exemplo, quando um estudante solicita o empréstimo de um notebook, o caso de uso responsável por essa operação verifica se o usuário está ativo, consulta a disponibilidade do equipamento e aplica as regras necessárias para registrar a solicitação.

Essas verificações devem ocorrer dentro do núcleo da aplicação, sem que ele precise conhecer diretamente o banco de dados ou a interface utilizada pelo estudante.

2- Portas e adaptadores

As portas representam os contratos de comunicação entre o núcleo da aplicação e os elementos externos. Os adaptadores implementam esses contratos ou permitem que sistemas externos acionem os casos de uso.

No SIGER, podemos identificar:

Adaptadores de entrada:

- Interface Web: permite que estudantes, professores, colaboradores e administradores utilizem o sistema.
- API REST: permite que outras aplicações ou clientes se comuniquem com as funcionalidades do SIGER.

Portas de saída:

- Porta de persistência: define como os casos de uso solicitam o armazenamento e a recuperação de informações.
- Porta de notificação: define como a aplicação solicita o envio de notificações.

Adaptadores de saída:

- Adaptador PostgreSQL: implementa a comunicação com o banco de dados.

- Adaptador de e-mail: realiza o envio de notificações por meio de um serviço externo.

A principal característica dessa organização é que o núcleo conhece as portas, mas não precisa conhecer as tecnologias utilizadas pelos adaptadores.

3- Exemplo de funcionamento no SIGER

Imagine que um estudante deseja reservar um projetor para uma apresentação acadêmica.

1. O estudante acessa a Interface Web e solicita a reserva.
2. A interface encaminha a solicitação ao caso de uso correspondente.
3. O caso de uso verifica as regras de negócio, como a situação do usuário e a disponibilidade do projetor.
4. O núcleo utiliza a porta de persistência para consultar ou registrar as informações necessárias.
5. O adaptador PostgreSQL executa as operações no banco de dados.

6. Caso a categoria do recurso exija aprovação, a solicitação fica aguardando a decisão de um administrador.
7. Quando houver uma decisão, o caso de uso poderá solicitar o envio de uma notificação por meio da porta de notificação.
8. O adaptador de e-mail realiza o envio utilizando o serviço externo.

Dessa forma, mesmo que futuramente o SIGER substitua o PostgreSQL por outro banco de dados ou utilize um serviço diferente para enviar notificações, as regras de negócio não precisarão ser reescritas por causa dessa mudança.

Benefício para o SIGER: maior independência tecnológica, facilidade de testes e flexibilidade para substituir ou integrar serviços externos.

Aplicação da Clean Architecture no SIGER

A **Clean Architecture (Arquitetura Limpa)** tem como principal objetivo organizar o sistema em camadas com responsabilidades bem definidas, garantindo que as regras de negócio permaneçam independentes de tecnologias externas.

Sua principal característica é a chamada **Regra de Dependência**: as dependências do código devem apontar para dentro, em direção às regras de negócio. As camadas internas não devem depender diretamente das camadas externas.

No SIGER, essa organização permite separar as entidades do domínio, os casos de uso, os mecanismos de comunicação e a infraestrutura tecnológica.

1- Entidades e regras de domínio

Essa é a camada mais interna da arquitetura. Ela representa os conceitos fundamentais do SIGER e suas regras de negócio.

Podemos encontrar entidades como:

- **Usuário:** representa estudantes, professores, colaboradores e administradores.
- **Recurso:** representa equipamentos e espaços disponíveis para empréstimo ou reserva.
- **Empréstimo:** representa a utilização temporária de um recurso.
- **Reserva:** representa a solicitação de utilização futura de um recurso.
- **Penalidade:** representa possíveis consequências de atrasos na devolução.

Essas entidades devem concentrar regras essenciais do domínio, como a impossibilidade de realizar operações incompatíveis com o estado atual de um empréstimo.

2- Casos de uso

A camada de casos de uso coordena as operações oferecidas pelo SIGER.

Entre suas responsabilidades estão:

- Solicitar empréstimos.
- Solicitar reservas.
- Consultar disponibilidade.
- Decidir solicitações.
- Registrar retiradas e devoluções.
- Calcular ou registrar penalidades.
- Consultar históricos.

Por exemplo, o caso de uso **Solicitar Empréstimo** coordena a validação do usuário, a consulta da disponibilidade do recurso e o registro da solicitação.

Ele utiliza contratos para acessar informações persistidas e serviços externos, sem precisar conhecer diretamente a implementação do banco de dados ou do serviço de notificação.

3- Adaptadores de interface

Essa camada faz a comunicação entre os casos de uso e os elementos externos.

No SIGER, pode conter:

- **Controladores:** recebem as solicitações enviadas pela interface ou pela API e acionam os casos de uso correspondentes.
- **Adaptadores de persistência:** implementam os contratos utilizados pelos casos de uso para consultar e armazenar informações.
- **Adaptadores de notificação:** conectam os casos de uso aos serviços responsáveis pelo envio de mensagens.

Essa camada também pode converter dados recebidos externamente para formatos adequados aos casos de uso e preparar os resultados para serem apresentados ao usuário.

4- Frameworks e infraestrutura

É a camada mais externa, responsável pelas tecnologias e ferramentas utilizadas pelo sistema.

No SIGER, podemos considerar:

- Interface Web.
- API REST.
- Banco de dados PostgreSQL.
- Serviço externo de e-mail.
- Frameworks e bibliotecas utilizados no desenvolvimento.

Essas tecnologias podem ser substituídas ou atualizadas sem que as entidades e regras de negócio precisem depender diretamente delas.

5- Exemplo de funcionamento no SIGER

Imagine novamente que um estudante deseja solicitar o empréstimo de um notebook.

1. O estudante realiza a solicitação pela Interface Web.
2. A solicitação chega ao controlador responsável.
3. O controlador encaminha os dados ao caso de uso **Solicitar Empréstimo**.

4. O caso de uso utiliza as entidades e regras do domínio para validar a operação.
5. Os contratos de persistência são utilizados para consultar a disponibilidade e registrar a solicitação.
6. O adaptador de persistência realiza a comunicação com o PostgreSQL.
7. O resultado retorna ao caso de uso e, posteriormente, ao controlador.
8. O controlador prepara a resposta para a interface apresentar ao estudante.

A comunicação com o banco de dados ocorre por meio de abstrações, respeitando a regra de dependência da Clean Architecture.

Benefício para o SIGER: maior organização do código, separação de responsabilidades, facilidade de manutenção e possibilidade de testar as regras de negócio sem depender da infraestrutura.

Validação das arquiteturas em cenários de qualidade

1- Cenário de manutenibilidade e modificabilidade:

Situação: a universidade decide substituir o serviço de envio de e-mails por uma plataforma de notificações diferente.

Resposta esperada: o sistema deve permitir a substituição do serviço sem exigir alterações nas principais regras de negócio, como solicitar empréstimos, aprovar reservas ou registrar devoluções.

Arquitetura Hexagonal: esse cenário é favorecido pela utilização de portas e adaptadores. O núcleo da aplicação depende de uma porta de notificação, enquanto o adaptador de saída implementa a comunicação com o serviço externo. Assim, a substituição do serviço exige principalmente a criação ou alteração do adaptador.

Clean Architecture: a separação entre casos de uso, adaptadores de interface e infraestrutura também favorece essa mudança. O caso de uso utiliza uma abstração para solicitar notificações, enquanto o adaptador realiza a comunicação com o serviço externo.

Avaliação: ambas apresentam bom comportamento nesse cenário, pois evitam que as regras de negócio fiquem diretamente acopladas a uma tecnologia de notificação.

2- Cenário de segurança

Situação: um estudante tenta acessar uma funcionalidade administrativa, como gerenciar recursos ou aprovar solicitações de empréstimo.

Resposta esperada: o sistema deve verificar as permissões do usuário e impedir operações não autorizadas, independentemente da interface utilizada para acessar a funcionalidade.

Arquitetura Hexagonal: os casos de uso e o núcleo da aplicação podem concentrar validações importantes de autorização, evitando que a segurança dependa exclusivamente da interface Web. Adaptadores de entrada diferentes podem acionar os mesmos casos de uso, que devem respeitar as regras de acesso.

Clean Architecture: a separação entre controladores, casos de uso e entidades permite organizar a autenticação e a autorização em pontos apropriados da aplicação. As regras de acesso podem ser verificadas antes da execução de operações administrativas.

Avaliação: ambas permitem estruturar mecanismos de segurança de maneira organizada. Entretanto, nenhuma arquitetura garante segurança automaticamente. É necessário implementar autenticação, autorização, validação de entradas, proteção de dados e registro de operações relevantes.

3-Cenário de testabilidade

Situação: a equipe de desenvolvimento precisa testar a funcionalidade de solicitação de empréstimo sem realizar conexões com o banco de dados real ou enviar notificações verdadeiras.

Resposta esperada: deve ser possível verificar as regras de negócio utilizando dados controlados e simulações dos serviços externos.

Arquitetura Hexagonal: as portas permitem substituir os adaptadores reais por implementações simuladas durante os testes. Dessa forma, é possível testar o núcleo da aplicação isoladamente, verificando, por exemplo, se um usuário inativo é impedido de solicitar um empréstimo.

Clean Architecture: a separação das entidades e dos casos de uso em relação à infraestrutura permite testar as regras e os fluxos de aplicação sem depender diretamente do PostgreSQL, da interface Web ou de serviços externos.

Avaliação: as duas arquiteturas favorecem a testabilidade. A Hexagonal destaca a substituição de dependências externas por adaptadores de teste, enquanto a Clean Architecture destaca a independência das camadas internas.

Trade-offs das arquiteturas

Trade-offs da Arquitetura Hexagonal

Vantagens:

- Isola as regras de negócio de tecnologias externas, como bancos de dados e serviços de notificação.
- Facilita substituir integrações sem modificar diretamente o núcleo da aplicação.
- Permite realizar testes utilizando adaptadores simulados.
- Favorece a integração com novas APIs e serviços externos.
- Mantém a aplicação mais independente de ferramentas e frameworks específicos.

Desvantagens e custos:

- Exige planejamento para definir corretamente as portas e os adaptadores.
- Pode aumentar a quantidade de interfaces, classes e componentes do projeto.
- Para sistemas pequenos, essa separação pode representar complexidade desnecessária.

- A equipe precisa compreender bem os limites entre o núcleo da aplicação e os componentes externos.

Impacto no SIGER: a arquitetura é interessante porque o sistema pode precisar integrar diferentes serviços ao longo do tempo, como plataformas de e-mail, sistemas de autenticação institucional e outros mecanismos de persistência.

Trade-offs da Clean Architecture

Vantagens:

- Organiza o sistema em camadas com responsabilidades bem definidas.
- Mantém as regras de negócio independentes de frameworks e infraestrutura.
- Facilita a manutenção e a evolução das funcionalidades.
- Favorece testes isolados das entidades e dos casos de uso.
- Ajuda a manter uma estrutura organizada conforme o sistema cresce.

Desvantagens e custos:

- Exige planejamento para definir corretamente as responsabilidades de cada camada.
- Pode gerar uma quantidade maior de classes, interfaces e conversões de dados.
- A implementação inicial pode ser mais trabalhosa do que uma estrutura mais simples.
- Se as camadas forem organizadas incorretamente, podem surgir dependências indevidas e acoplamento entre componentes.

Impacto no SIGER: a arquitetura é interessante porque o sistema possui diferentes funcionalidades e regras de negócio, como empréstimos, reservas, aprovações, devoluções, penalidades e gerenciamento de usuários. A separação em camadas pode facilitar a organização e a evolução dessas responsabilidades.